

LINUX PROCESS

Command Line Arguments of a Process

1. Write a C program that accepts multiple filenames as command line arguments and prints the size of each file.

PROGRAM:

```
/*
Program to display the size of files provided as command-line
arguments
Author   : MUTHUGANESH S
Date     : 08/03/2026
Filename : filesize.c
*/

#include <stdio.h>

int main(int argc, char *argv[])
{
    FILE *fp;
    long size;
    int i;

    if(argc < 2)
    {
        printf("Usage: %s filename1 filename2 ...\n", argv[0]);
        return 1;
    }

    for(i = 1; i < argc; i++)
    {
        fp = fopen(argv[i], "r");

        if(fp == NULL)
        {
            printf("Cannot open file %s\n", argv[i]);
            continue;
        }

        fseek(fp, 0, SEEK_END);
        size = ftell(fp);

        printf("File: %s Size: %ld bytes\n", argv[i], size);

        fclose(fp);
    }
}
```

```
    return 0;
}
```

OUTPUT:

```
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/LINUX PROCESS"
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc filesize.c -o filesize
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./filesize
Usage: ./filesize filename1 filename2 ...
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$
```

2. Implement a program that parses command line arguments to perform different operations (e.g., add, subtract, multiply) on two integers.

PROGRAM:

```
/*
Program to perform basic arithmetic operations based on
command-line arguments.
Author   : MUTHUGANESH S
Date     : 05/03/2026
Filename : cal.c
*/

//header files
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
    int a, b;

    if(argc != 4)
    {
        printf("Usage: %s operation num1 num2\n", argv[0]);
        return 1;
    }

    a = atoi(argv[2]);
    b = atoi(argv[3]);

    if(strcmp(argv[1], "add") == 0)
        printf("Result = %d\n", a + b);
    else if(strcmp(argv[1], "sub") == 0)
        printf("Result = %d\n", a - b);
    else if(strcmp(argv[1], "mul") == 0)
```

```

        printf("Result = %d\n", a * b);
    else
        printf("Invalid operation\n");

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc cal.c -o cal
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./cal
Usage: ./cal operation num1 num2
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ |

```

Environment of a Process

1. Write a C program that prints all environment variables of the current process.

PROGRAM:

```

/*
Program to print environment variables
Author   : MUTHUGANESH S
Date     : 07/03/2026
Filename : env.c
*/

#include <stdio.h>

// Global variable that holds the environment variables
extern char **environ;

int main()
{
    char **env = environ;

    while(*env != NULL)
    {
        printf("%s\n", *env);
        env++;
    }

    return 0;
}

```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc env.c -o env
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./env
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
TRAINING_PATH=/mnt/m/training/LINUX
WSL_DISTRO_NAME=Ubuntu
NAME=MG
PWD=/mnt/m/training/LINUX/LINUX PROCESS
LOGNAME=mg
HOME=/home/mg
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/386_interop
WAYLAND_DISPLAY=wayland-0
TERM=xterm-256color
USER=mg
DISPLAY=:0
SHLVL=1
XDG_RUNTIME_DIR=/run/user/1000/
WSLENV=
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/desktop
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux.2.4.10.0_x64__8wekyb3d8bbwe:/mnt/c/Program Files/Java/jdk-21/bin:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/dotnet:/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/apache-maven-3.9.9/bin:/mnt/c/Program Files/MySQL/MySQL Server 8.0/bin:/mnt/c/MinGW/bin:/mnt/c/Program Files/nodejs:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/Git/bin:/mnt/c/apache-maven-3.9.11/bin:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/MUTHUGANESH S/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/MUTHUGANESH S/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/MUTHUGANESH S/AppData/Roaming/npm:/mnt/c/apache-maven-3.9.11/bin:/snap/bin
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
HOSTTYPE=x86_64
PULSE_SERVER=unix:/mnt/wslg/PulseServer
OLDPWD=/home/mg
_=./env
```

2. Implement a program that sets a custom environment variable and then retrieves its value using `getenv()`.

PROGRAM:

```
/*
Program to set and get environment variables in C.
Author   : MUTHUGANESH S
Date     : 07/03/2026
Filename : set.c
*/

//header files
#include <stdio.h>
#include <stdlib.h>

int main()
{
    setenv("MY_VAR", "HelloEnvironment", 1);

    char *value = getenv("MY_VAR");

    if(value != NULL)
        printf("MY_VAR = %s\n", value);
    else
        printf("Environment variable not found\n");

    return 0;
}
```

```
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc set.c -o set
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./set
MY_VAR = HelloEnvironment
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ |
```

fork

1. Write a C program that creates a child process using fork() and prints the process IDs of both parent and child.

PROGRAM:

```
/*
Program to implement fork()
Author   : MUTHUGANESH S
Date     : 08/03/2026
Filename : atomic.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>

int main() {
    // Create a child process using fork()
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process:\n");
        printf("PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }
    else {
        // Parent process
```

```

        printf("Parent Process:\n");
        printf("PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc fork.c -o fork
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./fork
Parent Process:
PID: 798
Child PID: 799
Child Process:
PID: 799
Parent PID: 445

```

2. Implement a program where the parent and child process each write to a shared file after fork(). Analyze the output.

PROGRAM:

```

/*
Program to implement shared log file
Author   : MUTHUGANESH S
Date     : 08/03/2026
Filename : shar.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("shared.txt", O_WRONLY | O_CREAT | O_APPEND,
0644);

    if (fd < 0) {

```

```

        printf("File open error\n");
        return 1;
    }

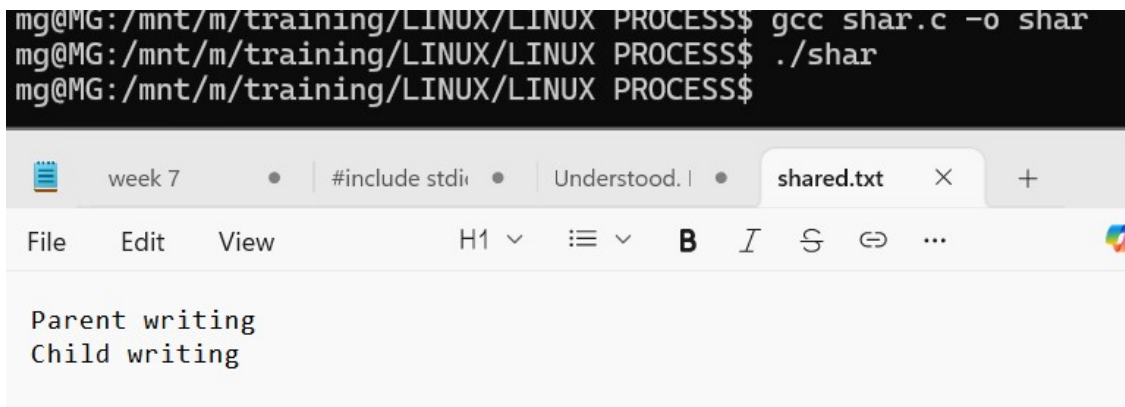
    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        write(fd, "Child writing\n", 14);
    }
    else {
        // Parent process
        write(fd, "Parent writing\n", 15);
    }

    close(fd);
    return 0;
}

```

OUTPUT:



The screenshot shows a terminal window at the top with the following commands and output:

```

mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc shar.c -o shar
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./shar
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$

```

Below the terminal is a text editor window titled "shared.txt". The editor's content area displays the output of the program:

```

Parent writing
Child writing

```

exec

1. Write a C program that uses `execvp()` to replace the current process image with the `ls` command.

PROGRAM:

```
/*
Program to implement execvp()
Author   : MUTHUGANESH S
Date     : 08/03/2026
Filename : exe.c
*/

//Header files
#include <stdio.h>
#include <unistd.h>

int main() {
    char *args[] = {"ls", "-l", NULL};

    printf("Before execvp()\n");

    // This line executes ONLY if execvp() succeeds
    execvp("ls", args);

    // This line executes ONLY if execvp fails
    perror("execvp failed");
    return 1;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc exe.c -o exe
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./exe
Before execvp()
total 1148
drwxrwxrwx 1 mg mg    4096 Mar 16 19:24 'COMMAND LINE'
drwxrwxrwx 1 mg mg    4096 Mar 16 19:24 'ENVIRONMENT PROCESS'
drwxrwxrwx 1 mg mg    4096 Mar 17 10:48 EXEC
drwxrwxrwx 1 mg mg    4096 Mar 17 10:48 FORK
-rwxrwxrwx 1 mg mg 1156341 Mar 16 19:25 'LINUX PROCESS.docx'
-rwxrwxrwx 1 mg mg   16096 Mar 17 10:50 exe
-rwxrwxrwx 1 mg mg    433 Mar 17 10:50 exe.c
-rwxrwxrwx 1 mg mg    162 Mar 17 10:38 '~$NIX PROCESS.docx'
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ |
```


2. Implement a program that forks a child and uses `execlp()` to run a different program, passing command line arguments.

PROGRAM:

```
/*
Program to implement fork() and execlp()
Author   : MUTHUGANESH S
Date     : 08/03/2026
Filename : fe.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child: executing 'ls -a'\n");

        execlp("ls", "ls", "-a", NULL);

        // Executes only if execlp fails
        perror("execlp failed");
        return 1;
    }
    else {
        // Parent process
        wait(NULL);
        printf("Parent: child finished execution\n");
    }

    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc fe.c -o fe
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./fe
Child: executing 'ls -a'
.          EXEC          exe.c
..         FORK          fe
'COMMAND LINE'  'LINUX PROCESS.docx'  fe.c
'ENVIRONMENT PROCESS'  exe  '~$NIX PROCESS.docx'
Parent: child finished execution
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ |
```

wait and waitpid

1. Write a C program that creates multiple child processes and uses wait() to collect their exit statuses.

PROGRAM:

```
/*
Program to implement wait()
Author   : MUTHUGANESH S
Date     : 08/03/2026
Filename : wait.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int i;
    pid_t pid;

    for (i = 0; i < 3; i++) {
        pid = fork();

        if (pid == 0) {
            // Child process
            printf("Child %d: PID = %d\n", i, getpid());
            exit(10 + i); // distinct exit codes
        }
    }

    // Parent process
```

```

int status;
pid_t wpid;

while ((wpid = wait(&status)) > 0) {
    if (WIFEXITED(status)) {
        printf("Parent: Child with PID %d exited with
status %d\n",
                wpid, WEXITSTATUS(status));
    }
}

return 0;
}

```

OUTPUT:

```

Parent: Child finished execution
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc wait.c -o wait
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./wait
Child 0: PID = 4555
Child 1: PID = 4556
Parent: Child with PID 4555 exited with status 10
Parent: Child with PID 4556 exited with status 11
Child 2: PID = 4557
Parent: Child with PID 4557 exited with status 12
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$

```

2. Implement a program that uses waitpid() to wait for a specific child process to finish and prints its exit code.

PROGRAM:

```

/*
Program to implement waitpid()
Author   : MUTHUGANESH S
Date     : 08/03/2026
Filename : waitpid.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

```

```

int main() {
    pid_t pid1, pid2;
    int status;

    pid1 = fork();

    if (pid1 == 0) {
        // First child
        sleep(2);
        printf("Child 1 done (PID=%d)\n", getpid());
        exit(21);
    }

    pid2 = fork();

    if (pid2 == 0) {
        // Second child
        sleep(1);
        printf("Child 2 done (PID=%d)\n", getpid());
        exit(42);
    }

    // Parent process
    printf("Parent waiting specifically for child PID %d\n",
pid1);

    waitpid(pid1, &status, 0);

    if (WIFEXITED(status)) {
        printf("Parent: Child %d exited with status %d\n",
            pid1, WEXITSTATUS(status));
    }

    // Clean up remaining child
    wait(NULL);

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc waitpid.c -o waitpid
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./waitpid
Parent waiting specifically for child PID 4565
Child 2 done (PID=4566)
Child 1 done (PID=4565)
Parent: Child 4565 exited with status 21
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ |

```

Orphan Process and Zombie Process

1. Write a C program that creates an orphan process and demonstrates its adoption by the init process.

PROGRAM:

```
/*
Program to demonstrate orphan process creation in C.
Author   : MUTHUGANESH S
Date     : 09/03/2026
Filename : orp.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        sleep(3); // ensure parent exits first

        printf("Child: PID = %d\n", getpid());
        printf("Child: New Parent PID = %d\n", getppid());

        // Infinite loop to observe via ps
        while (1) {
            sleep(1);
        }
    }
    else {
        // Parent process
        printf("Parent: PID = %d\n", getpid());
        printf("Parent exiting...\n");
        exit(0);
    }

    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc orp.c -o orp
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./orp
Parent: PID = 4576
Parent exiting...
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ Child: PID = 4577
Child: New Parent PID = 445
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$
```

2. Implement a program that creates a zombie process and shows how it appears in the process table until the parent calls wait().

PROGRAM:

```
/*
Program to demonstrate Zombie process creation in C.
Author   : MUTHUGANESH S
Date     : 09/03/2026
Filename : zom.c
*/

// Include necessary header files
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child exiting (PID=%d)\n", getpid());
        exit(99);
    }
    else {
        // Parent process
        printf("Parent sleeping (PID=%d)\n", getpid());
        sleep(10); // Delay wait()

        // Uncomment to clean zombie:
        // wait(NULL);
    }
}
```

```

        printf("Parent exiting\n");
    }

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc zom.c -o zom
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./zom
Parent sleeping (PID=4584)
Child exiting (PID=4585)

Parent exiting
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$

```

Signals

1. Write a C program that handles SIGINT (Ctrl+C) and prints a message before exiting.

PROGRAM:

```

/*
Process Signal Handling Example in C
Author   : MUTHUGANESH S
Date     : 09/03/2026
Filename : sig.c
*/

// Include necessary header files
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void handle_sigint(int sig) {
    printf("\nCaught SIGINT (signal %d)\n", sig);
    printf("Exiting gracefully...\n");
    exit(0);
}

int main() {
    signal(SIGINT, handle_sigint);

    printf("Press Ctrl+C to trigger SIGINT...\n");

    while (1) {

```

```

        sleep(1);
    }

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc sig.c -o sig
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./sig
Press Ctrl+C to trigger SIGINT...
^C
Caught SIGINT (signal 2)
Exiting gracefully...
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$

```

2. Implement a program that sends SIGUSR1 to another process and handles it with a custom signal handler.

PROGRAM:

```

/*
Program to send a signal to another process using kill()
system call
Author: MUTHUGANESH S
Date      : 09/03/2026
Filename  : sig_s.c
*/

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    pid_t pid;

    printf("Enter receiver PID: ");
    scanf("%d", &pid);

    if (kill(pid, SIGUSR1) == 0) {
        printf("SIGUSR1 sent successfully\n");
    } else {
        perror("kill failed");
    }

    return 0;
}

```



```

/*
Program to send a signal to another and handle it
Author: MUTHUGANESH S
Date      : 09/03/2026
Filename  : sig_r.c
*/

// Include necessary header files
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_sigusr1(int sig) {
    printf("Received SIGUSR1 (signal %d)\n", sig);
}

int main() {
    signal(SIGUSR1, handle_sigusr1);

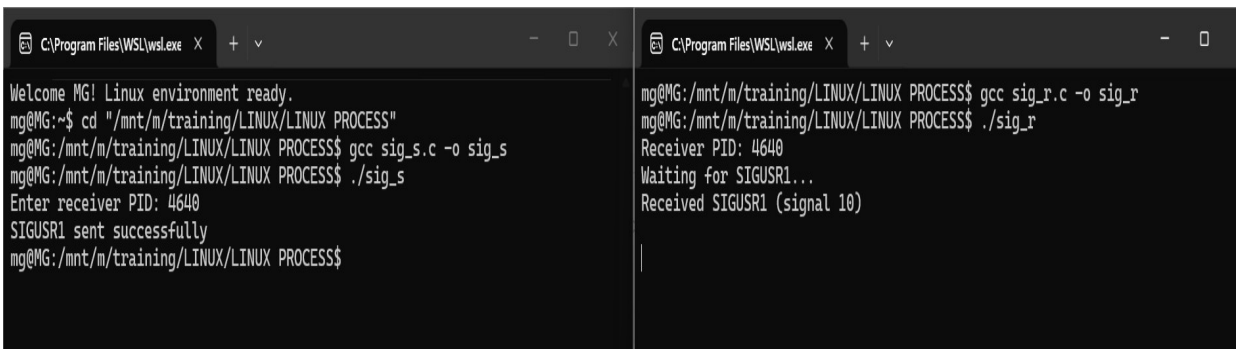
    printf("Receiver PID: %d\n", getpid());
    printf("Waiting for SIGUSR1...\n");

    while (1) {
        pause(); // wait for signal
    }

    return 0;
}

```

OUTPUT:



```

C:\Program Files\WSL\wsl.exe X + v
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/LINUX PROCESS"
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc sig_s.c -o sig_s
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./sig_s
Enter receiver PID: 4640
SIGUSR1 sent successfully
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$

C:\Program Files\WSL\wsl.exe X + v
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc sig_r.c -o sig_r
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./sig_r
Receiver PID: 4640
Waiting for SIGUSR1...
Received SIGUSR1 (signal 10)

```

Signal Ignore, Signal Default

1. Write a C program that ignores SIGINT and demonstrates that pressing Ctrl+C does not terminate the process.

PROGRAM:

```

/*
Program to ignore SIGINT signalprocess
Author   : MUTHUGANESH S
Date     : 10/03/2026
Filename : sig_i.c
*/

// Include necessary header files
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    // Ignore SIGINT
    signal(SIGINT, SIG_IGN);
    printf("SIGINT is ignored. Press Ctrl+C...\n");
    while (1) {
        printf("Running...\n");
        sleep(2);
    }
    return 0;
}

```

OUTPUT:

[illegible]

2. Implement a program that sets the signal handler for SIGTERM to default, then sends SIGTERM to itself and observes the result.

PROGRAM:

```
/*
Program to set default action for SIGTERM signal
Author   : MUTHUGANESH S
Date     : 10/03/2026
Filename : sig_d.c
*/

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    printf("PID: %d\n", getpid());

    // Set SIGTERM to default action
    signal(SIGTERM, SIG_DFL);

    printf("Sending SIGTERM to self...\n");

    sleep(2); // delay for observation

    kill(getpid(), SIGTERM);

    // This line will NOT execute if default action occurs
    printf("This line should not be printed\n");

    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ gcc sig_d.c -o sig_d
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$ ./sig_d
PID: 4709
Sending SIGTERM to self...
Terminated
mg@MG:/mnt/m/training/LINUX/LINUX PROCESS$
```